

ON EMIT

Gmail + Google Calendar workflow command center

Stack	Next.js 15, Supabase Auth/Postgres, Corsair, tRPC
Integrations	Gmail and Google Calendar through Corsair
Repository	github.com/KOUSHIKG04/on-emit
Final branch	codex/final-documentation
Verification	TypeScript and production build passed

Outcome

The application now uses live, tenant-isolated Gmail and Calendar data. It adds full MIME email reading, replies and drafts, Calendar updates, advanced Gmail search, priority filtering, realtime webhook refresh, keyboard workflows, and confirmed Corsair MCP agent actions.

Generated 20 July 2026

1. Architecture and identity flow

Supabase authenticates the human. The validated Supabase subject claim becomes the Corsair tenant ID. Browser input never chooses a tenant ID.

1. Browser	2. Next.js middleware	3. Supabase
Sends Supabase cookies. UI calls tRPC and protected routes.	Refreshes cookies with getClaims() and returns the exact updated response.	Validates JWT claims. The sub claim identifies the workspace user.
4. tRPC server	5. Corsair tenant	6. Google APIs
Protected procedure reads ctx.userId from validated claims.	corsair.withTenant(ctx.userId) isolates tokens, cache, email, and events.	Gmail and Calendar operations run through Corsair plugins.

Middleware responsibility

- Read all cookies from NextRequest.
- Create the Supabase server client with request/response cookie adapters.
- Call supabase.auth.getClaims() so expired access tokens can be refreshed.
- Return the same NextResponse carrying refreshed Set-Cookie headers.
- It does not connect Gmail or call Corsair; protected server procedures do that after authentication.

Separation of OAuth roles

System	Purpose	Configuration location
Supabase Auth	Signs users into On Emit	Supabase Auth URL settings
Corsair OAuth	Connects each user's Google account	Corsair Gmail/Calendar integration credentials
Supabase OAuth Server	Makes Supabase an identity provider for third parties	Not required for this project

2. Implemented workflows

Area	Implementation	Workflow gain
Email reader	RFC/MIME parsing, sanitized iframe HTML, Markdown/plain text, CID images, opt-in remote images, authenticated attachments	Displays transactional, newsletter, image, text, and attachment-heavy email safely
Gmail actions	Reply with thread headers, save draft, archive, read/unread	Actions remain inside the thread reader and have shortcuts
Advanced search	Corsair Gmail threads.list with native Gmail q syntax	Search by from:, subject:, date, unread, starred, and attachments
Priority inbox	Latest subject/body classified in one model batch and cached per tenant; local rule fallback	Surfaces action-required mail and explains each label
Calendar	Create, edit, cancel, attendee updates, timed and all-day events	Meeting changes no longer require opening Google Calendar
Realtime	Corsair processWebhook plus Postgres LISTEN/NOTIFY and authenticated SSE	New provider events invalidate local UI caches without Google polling
Agent	Corsair MCP OpenAI Agents adapter with preview/confirm phases	One request can combine email and Calendar operations safely

Keyboard map

Keys	Action
Ctrl/Cmd+K	Open quick action
/	Focus advanced Gmail search
R	Reply
E	Archive
Shift+I / Shift+U	Mark read / unread
Ctrl/Cmd+Enter	Send reply

3. Realtime webhook flow

Corsair verifies and routes provider payloads. On Emit adds a tenant HMAC and publishes only a small invalidation event; the browser then reloads authoritative data through protected tRPC procedures.

Step	Major code block
1	Google Gmail Pub/Sub or Calendar watch sends POST to <code>/api/webhooks/corsair</code>
2	<code>tenant-token.ts</code> verifies the HMAC in constant time
3	<code>processWebhook(corsair, headers, body, { tenantId })</code> matches, verifies, handles, and caches
4	Postgres NOTIFY publishes <code>tenantId</code> , <code>plugin</code> , <code>action</code> , and <code>timestamp</code>
5	<code>/api/realtime</code> authenticates Supabase and listens for only that tenant
6	<code>RealtimeSync</code> invalidates Gmail, Calendar, or integration React Query caches

Provider-side activation

- Deploy the app or expose localhost with ngrok over HTTPS.
- Set `APP_URL` to that public origin and reconnect integrations if callback URLs changed.
- Copy the protected webhook URL shown in Connected services.
- Complete Corsair Gmail Pub/Sub topic and Calendar watch configuration from the setup videos.
- Send a test email and test invite, then confirm immediate UI refresh.

Important deployment note

Server-sent events need a host that supports long-lived Node.js responses. Postgres notifications work across multiple app instances, but a platform with short serverless execution limits may require a managed realtime transport.

4. Database, AI, and security

Database ownership

Table	Purpose
<code>corsair_integrations</code>	Provider integration configuration
<code>corsair_accounts</code>	Encrypted tenant-scoped provider accounts
<code>corsair_entities</code>	Corsair cached provider entities
<code>corsair_events</code>	Corsair webhook/event records
<code>corsair_email_priorities</code>	On Emit classification cache keyed by tenant and thread

AI behavior

- Priority classification uses the cost-sensitive model configured by `OPENAI_PRIORITY_MODEL`; default `gpt-5.6-luna`.
- Agent chat uses `OPENAI_AGENT_MODEL`; default `gpt-5.6-terra`.
- If `OPENAI_API_KEY` is absent, priority filtering uses explainable local rules and agent chat reports setup required.
- Email content is treated as untrusted data and model prompts explicitly forbid following instructions inside messages.
- Agent write tools are removed from the preview run and exposed only after Confirm and execute.

Security controls

- Server-derived tenant identity from validated Supabase claims.
- Corsair tokens remain encrypted in Postgres with `CORSAIR_KEK`.
- Sanitized email HTML in a sandboxed iframe; remote images blocked by default.
- Attachment route revalidates session and tenant before Gmail download.
- Webhook HMAC comparison uses `timingSafeEqual`.
- Safe error messages are returned to the browser while detailed failures stay server-side.

5. Branch and verification report

Branch	Commit	Result
codex/fix-email-rendering	e4d2a60	Pushed
codex/gmail-actions	bb0dad1	Pushed
codex/calendar-management	1ffeb46	Pushed
codex/advanced-gmail-search	81304ff	Pushed
codex/priority-inbox	14dde9f	Pushed; table applied to Supabase
codex/realtime-webhooks	1081955	Pushed
codex/corsair-agent-chat	c65916d	Pushed
codex/final-documentation	5a2abf2	Pushed

Automated verification

- `npx tsc --noEmit` passed after each major feature.
- `npm run build` passed on the final application state.
- Drizzle generated the additive priority migration.
- `pnpm db:push` compared the live Supabase schema and applied the missing table.
- All feature branches were committed and pushed to origin.

Manual acceptance test

Test	Expected result
Sign in and connect	Supabase session; Gmail and Calendar both Connected
Open varied email	Sender, images, layouts, text and attachments render safely
Reply/draft/action	Google state changes and inbox refreshes
Search	Gmail operators return live connected-account threads
Calendar update	Event and attendee notifications update
Webhook	New email/invite refreshes UI without manual reload
Agent preview	No write occurs; plan asks for confirmation
Agent confirm	Corsair tools execute and report verified outcome

6. Deployment and submission checklist

Deployment

- Provision deployment environment variables from .env.example.
- Set APP_URL to the final HTTPS origin.
- Add the final /auth/callback URL in Supabase Auth URL Configuration.
- Set the final /api/corsair/oauth/callback URL in Corsair/Google OAuth configuration.
- Run `pnpm db:push` for this existing Supabase database.
- Deploy to a Node.js host that supports SSE, then reconnect Google services.
- Configure provider webhooks and perform the manual acceptance test.

Submission artifacts

Artifact	Status / action
GitHub repo	Ready: github.com/KOUSHIKG04/on-emit
Live link	Deploy and add URL to README
Demo video	Record the acceptance flow and add link
X/Twitter post	Publish and add link
LinkedIn post	Publish and add link
Short README	Completed
Corsair features list	Completed in README and this report
Bonus tasks list	Completed in README and this report

Recommended demo sequence

- Sign in, show connected tenant-isolated services, and open a real email with images/attachment.
- Use `/` to search with `has:attachment newer_than:30d`, then reply with `Ctrl/Cmd+Enter`.
- Filter Priority and show the reason/source tooltip.
- Create and edit a Calendar invite.
- Use agent chat to preview a combined invite + email, then confirm execution.
- Send a test message externally and show realtime arrival.

Repository: <https://github.com/KOUSHIKG04/on-emit>